TERRAFORM-BASED CONTAINERIZED MICROSERVICE DEPLOYMENT ON AWS

Implementation Report

Project detail	Value
Cloud provider	Amazon Web Services (AWS)
Infrastructure as Code	Terraform
Containerization	Docker
Container registry	Amazon Elastic Container Registry (ECR)
Compute	Amazon EC2 — Amazon Linux 2023
Load balancing	Application Load Balancer
AWS Region	us-east-1
Project directory	terraform-microservice

Prepared from the actual implementation and verification sequence

Document Guide

This document records the implementation in the order it was performed. It includes the Terraform configuration, Docker application files, shell scripts, commands, verification evidence and screenshot insertion points. Insert your own screenshots in the marked locations.

Note: The private SSH key and the real terraform.tfvars file contain sensitive configuration. Do not include private keys or expose personal IP addresses in a public submission.

Contents: Objective; Purpose; Prerequisites; Architecture; Project structure; implementation stages; final verification; concepts learned; conclusion; consolidated cleanup.

1. Objective

The objective of this assignment is to design and deploy a containerized microservice on AWS using Terraform. The application is a simple static web application served by Nginx inside a Docker container.

The complete infrastructure is provisioned and managed using Terraform, including:

- Amazon ECR private repository.
- Custom VPC and networking infrastructure.
- Two public subnets in separate Availability Zones.
- Internet Gateway and public route table.
- EC2 instance.
- IAM role and instance profile.
- SSH key pair imported into AWS.
- Application Load Balancer, target group and listener.
- Security groups controlling ALB, application and SSH traffic.

The Docker image is built locally, pushed to Amazon ECR through Terraform, and subsequently pulled by the EC2 instance to run the application. The application is accessed through the ALB; direct access to the EC2 application port is blocked.

2. Purpose

The purpose is to understand how Terraform automates deployment of a containerized application on AWS and how AWS services work together in a small DevOps deployment.

- Organize Terraform code using reusable modules.
- Build and test Docker images locally.
- Store container images in Amazon ECR.
- Provision AWS networking and compute resources using Terraform.
- Configure security groups and restricted SSH access.
- Use an EC2 IAM role to pull images from ECR without storing AWS access keys on the instance.
- Use EC2 user data to install Docker and deploy the application automatically.
- Expose the application through an Application Load Balancer.
- Verify application availability and network security.

3. Prerequisites and Environment

3.1 Local environment

Tool	Purpose
Windows	Local operating system.
Git Bash	Run Terraform, AWS CLI, Docker and SSH commands.
Visual Studio Code	Edit application, Terraform and script files.
Docker Desktop	Build and test the Docker image locally.
Terraform	Provision and manage AWS infrastructure.
AWS CLI	Authenticate and verify AWS resources.
SSH	Connect to the EC2 instance.

3.2 AWS configuration

Configuration	Value
AWS Region	us-east-1
Project name	terraform-microservice
VPC CIDR	10.0.0.0/16
Public subnet 1	10.0.1.0/24
Public subnet 2	10.0.2.0/24
EC2 instance type	t3.micro
Operating system	Amazon Linux 2023
ECR repository	terraform-microservice
Docker image tag	v1
Application/ALB port	80
SSH port	22

The SSH inbound rule was configured with the administrator's public IP address and a /32 CIDR. The IP value is intentionally represented as a placeholder in this document; use the actual authorized address in terraform.tfvars.

3.3 AWS authentication

AWS CLI authentication was checked before provisioning:

```
aws sts get-caller-identity
```

This command confirms the active AWS identity and account.

4. Architecture

4.1 Architecture overview

```
LOCAL WINDOWS MACHINE
  application/index.html + Dockerfile
                |
                v
          Docker Desktop
                |
                v
  Local image: terraform-microservice:v1
                |
                | Terraform local-exec:
                | ECR login, tag and push
                v
       Amazon ECR private repository
                |
                | EC2 IAM role permits image pull
                v
AWS CLOUD
  Custom VPC: 10.0.0.0/16
       |
       +-- Internet Gateway
       |
       +-- Public route table (0.0.0.0/0 -> IGW)
       |
       +-- Public subnet 1: 10.0.1.0/24
       |
       +-- Public subnet 2: 10.0.2.0/24
```

```
                  |
                  v
         Internet-facing ALB
             HTTP :80
                  |
                  v
         Target group HTTP :80
                  |
                  v
         EC2 – Amazon Linux 2023
                  |
                Docker
                  |
            Nginx container
                  |
             Static HTML page
```

4.2 Application traffic flow

1. A user opens the ALB application URL in a browser.
2. The request reaches the internet-facing Application Load Balancer.
3. The ALB listener accepts HTTP traffic on port 80.
4. The ALB forwards the request to its target group.
5. The target group forwards traffic to EC2 on port 80.
6. Docker publishes the Nginx container on EC2 port 80.
7. Nginx serves the static HTML page.
8. The response returns through the ALB to the browser.

4.3 Security design

Resource	Port	Allowed source
ALB security group	80	0.0.0.0/0
EC2 security group	80	ALB security group only
EC2 security group	22	Administrator public IP /32 only

The EC2 instance is placed in a public subnet to avoid a NAT Gateway in this lab and to allow SSH from the restricted administrator IP. Its application port is nevertheless not open to the internet; only the ALB security group can reach EC2 port 80.

5. Project Directory Structure

```
terraform-microservice/
├── application/
│   ├── index.html
│   └── Dockerfile
├── scripts/
│   ├── install_docker.sh
│   └── deploy_container.sh
├── modules/
│   ├── networking/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── ecr/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
```

```
│   ├── compute/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── loadbalancer/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── provider.tf
├── versions.tf
├── variables.tf
├── locals.tf
├── main.tf
├── outputs.tf
├── terraform.tfvars
└── .gitignore

Local SSH files (created during implementation):
  terraform-microservice-key
  terraform-microservice-key.pub
```

Directory/file	Purpose
application/	Application HTML and Dockerfile.
scripts/	EC2 Docker installation and container deployment scripts.
modules/ecr/	Creates the ECR repository.
modules/networking/	Creates VPC, subnets, routes and security groups.
modules/compute/	Creates EC2, SSH key pair, IAM role and instance profile.
modules/loadbalancer/	Creates ALB, listener, target group and attachment.
provider.tf	Configures AWS provider.
versions.tf	Defines Terraform and provider version requirements.
variables.tf	Declares root input variables.
locals.tf	Defines reusable local values.
main.tf	Connects modules and local image-push operation.
outputs.tf	Displays useful deployment information.
terraform.tfvars	Supplies project-specific values.
.gitignore	Excludes Terraform state/plan files and SSH keys.

6. Implementation

Stage 1 — Create the application and Docker image

6.1 Create application HTML

File: application/index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Sunith Terraform Microservice</title>
</head>
<body>
    <h1>Hello from Terraform Microservice!</h1>
    <h2>Application successfully deployed.</h2>
    <p>Powered by Docker, Nginx, AWS and Terraform.</p>
</body>
</html>
```

This is the static web page served by Nginx.

6.2 Create Dockerfile

File: application/Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

FROM selects the lightweight Nginx Alpine image. COPY places the application file in the Nginx document root. EXPOSE documents the container port. The Nginx base image already contains its startup command, so an additional CMD was not needed.

6.3 Build the Docker image

```
docker build -t terraform-microservice:v1 ./application
```

6.4 Run the container locally

```
docker run -d --name terraform-microservice-test -p 8080:80 terraform-microservice:v1
```

The -d option runs in the background; --name assigns a container name; -p 8080:80 maps laptop port 8080 to container port 80.

Open http://localhost:8080 in a browser. The page should display:

```
Hello from Sunith Terraform Microservice!
Application successfully deployed.
Powered by Docker, Nginx, AWS and Terraform.
```

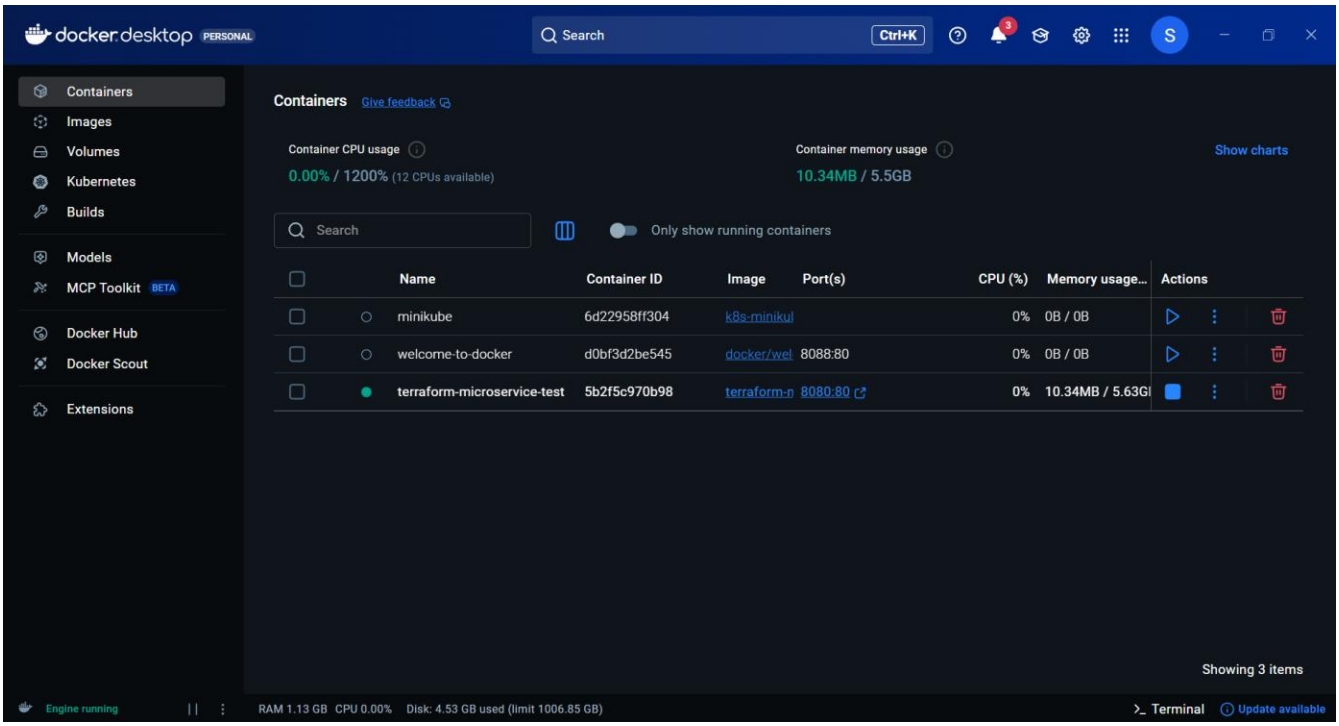
During initial local setup, Docker Desktop's Linux engine was not immediately available and a Docker Hub authentication error occurred. These issues were resolved before the local image build and browser test succeeded.

SCREENSHOT 01 — Local Docker Application Running



Capture: Browser displaying the application at http://localhost:8080.

SCREENSHOT 02 — Local Docker Container



Capture: Docker Desktop or terminal showing the local test container running.

Stage 2 — Configure Terraform and create Amazon ECR

6.5 Terraform version requirements

File: versions.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

6.6 AWS provider

File: provider.tf

```
provider "aws" {
  region = var.aws_region
}
```

6.7 Root variables

File: variables.tf

```
variable "aws_region" {
  description = "AWS deployment region"
  type        = string
}

variable "project_name" {
  description = "Project resource prefix"
  type        = string
}

variable "ssh_cidr" {
  description = "Your public IP allowed for SSH"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

6.8 Local values

File: locals.tf

```
locals {
  name_prefix = var.project_name
}
```

6.9 Terraform variable values

File: terraform.tfvars

```
aws_region   = "us-east-1"
project_name = "terraform-microservice"
ssh_cidr     = "YOUR_PUBLIC_IP/32"
```

Replace YOUR_PUBLIC_IP with the administrator's current public IP. The instance type uses the default t3.micro value.

6.10 ECR module

File: modules/ecr/main.tf

```
resource "aws_ecr_repository" "this" {
  name                 = var.repository_name
  image_tag_mutability = "MUTABLE"
  force_delete         = true

  image_scanning_configuration {
    scan_on_push = true
  }
}
```

File: modules/ecr/variables.tf

```
variable "repository_name" {
  type = string
}
```

File: modules/ecr/outputs.tf

```
output "repository_url" {
  value = aws_ecr_repository.this.repository_url
}
```

6.11 Connect the ECR module

Add this block to the root main.tf:

```
module "ecr" {
  source = "./modules/ecr"

  repository_name = local.name_prefix
}
```

6.12 Root ECR output

File: outputs.tf

```
output "ecr_repository_url" {
  description = "Amazon ECR repository URL"
  value       = module.ecr.repository_url
}
```

6.13 Initialize, format and validate Terraform

```
terraform init
terraform fmt -recursive
terraform validate
```

The validation returned: Success! The configuration is valid.

6.14 Plan and create ECR

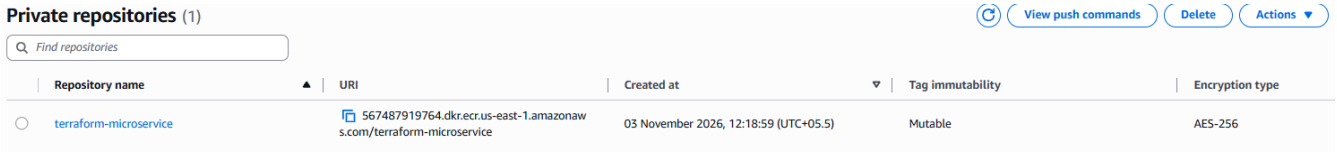
```
terraform plan
terraform apply
```

After reviewing the plan, type yes to approve. The ECR repository was created successfully.

Repository details observed during the implementation:

```
Repository name: terraform-microservice
Region: us-east-1
Repository URL:
567487919764.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice
```

SCREENSHOT 03 — Amazon ECR Repository Created



Capture: AWS Console showing the terraform-microservice private ECR repository.

Stage 3 — Push the local Docker image to ECR using Terraform

6.15 Terraform image-push resource

Add the following to the root main.tf, below the ECR module:

```
resource "terraform_data" "push_image" {
  triggers_replace = [
    filesha256("${path.module}/application/Dockerfile"),
    filesha256("${path.module}/application/index.html"),
    module.ecr.repository_url
  ]

  provisioner "local-exec" {
    interpreter = ["bash", "-c"]

    command = <<-EOT
      set -e

      aws ecr get-login-password --region ${var.aws_region} | docker login --username AWS --password-stdin
${split("/", module.ecr.repository_url)[0]}

      docker tag terraform-microservice:v1 ${module.ecr.repository_url}:v1

      docker push ${module.ecr.repository_url}:v1
    EOT
  }

  depends_on = [module.ecr]
}
```

The triggers_replace list tracks the Dockerfile, application HTML and ECR repository URL. local-exec runs on the machine executing Terraform. The commands authenticate Docker to ECR, tag the local image with the ECR repository URL and push it. The dependency ensures that the repository exists before the push.

6.16 Execute the image push

```
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
```

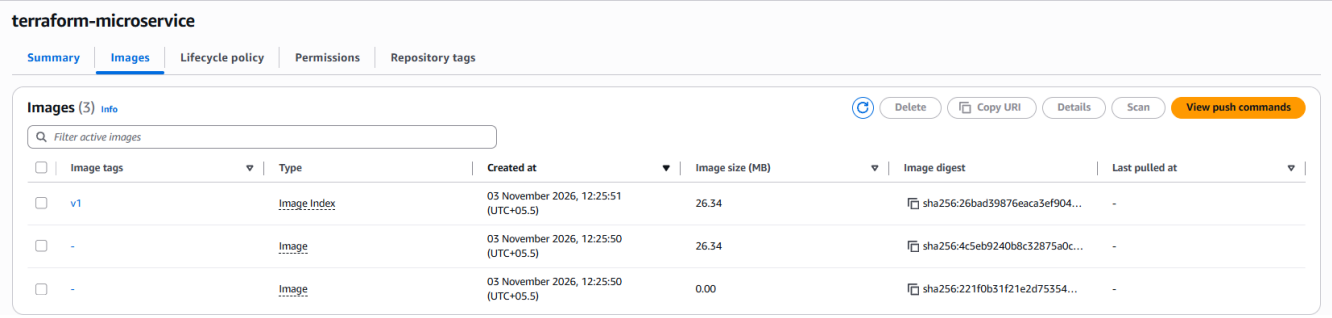
The push operation completed successfully. Terraform reported creation of terraform_data.push_image and Docker reported a successful push with a digest. A subsequent terraform apply reported no changes; this was expected because the trigger values had not changed.

6.17 Verify the ECR image

```
aws ecr describe-images \
  --repository-name terraform-microservice \
  --region us-east-1 \
  --no-cli-pager \
  --query 'imageDetails[].{Tags:imageTags,Digest:imageDigest,Status:imageStatus}' \
  --output table
```

The ECR image listing was checked for the v1 tag and ACTIVE image status.

SCREENSHOT 04 — Docker Image Pushed to Amazon ECR



Capture: AWS Console showing the terraform-microservice repository and uploaded v1 image tag.

Stage 4 — Create VPC and networking infrastructure

6.18 Networking module main.tf

```
data "aws_availability_zones" "available" {
  state = "available"
}

resource "aws_vpc" "this" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = { Name = "${var.project_name}-vpc" }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id

  tags = { Name = "${var.project_name}-igw" }
}

resource "aws_subnet" "public" {
  count = 2

  vpc_id                  = aws_vpc.this.id
  cidr_block              = "10.0.${count.index + 1}.0/24"
  availability_zone       = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
```

```
  tags = { Name = "${var.project_name}-public-${count.index + 1}" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }

  tags = { Name = "${var.project_name}-public-rt" }
}

resource "aws_route_table_association" "public" {
  count = 2

  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "alb" {
  name   = "${var.project_name}-alb-sg"
  vpc_id = aws_vpc.this.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "ec2" {
  name   = "${var.project_name}-ec2-sg"
  vpc_id = aws_vpc.this.id

  ingress {
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_cidr]
  }

  egress {
    from_port   = 0
```

```
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

6.19 Networking variables

File: modules/networking/variables.tf

```
variable "project_name" {
  type = string
}

variable "ssh_cidr" {
  type = string
}
```

6.20 Networking outputs

File: modules/networking/outputs.tf

```
output "vpc_id" {
  value = aws_vpc.this.id
}

output "subnet_ids" {
  value = aws_subnet.public[*].id
}

output "alb_security_group_id" {
  value = aws_security_group.alb.id
}

output "ec2_security_group_id" {
  value = aws_security_group.ec2.id
}
```

6.21 Connect networking module

Add this block to the root main.tf:

```
module "networking" {
  source = "./modules/networking"

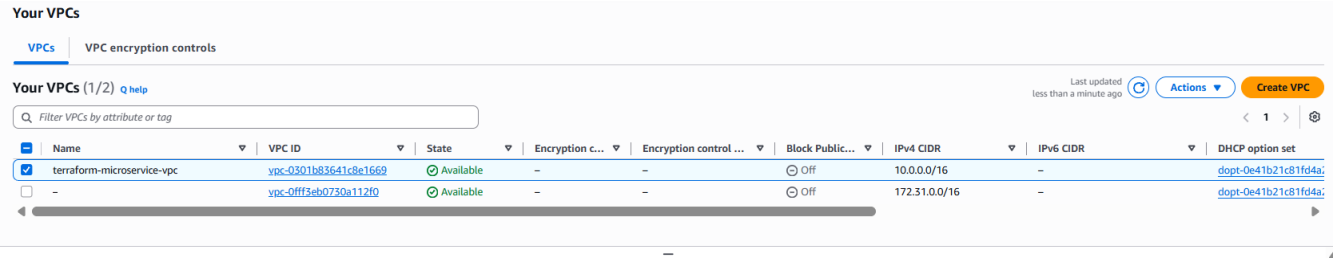
  project_name = local.name_prefix
  ssh_cidr     = var.ssh_cidr
}
```

6.22 Create networking resources

```
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
```

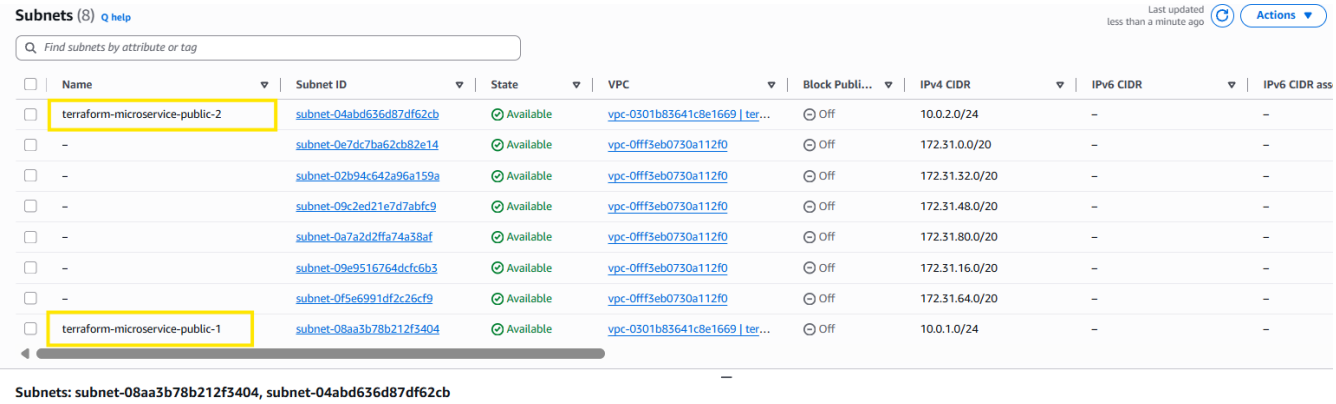
The networking resources created were one VPC, one Internet Gateway, two public subnets, one public route table, two route table associations, one ALB security group and one EC2 security group. The VPC and route table associations were checked in the AWS Console.

SCREENSHOT 05 — Custom VPC Created



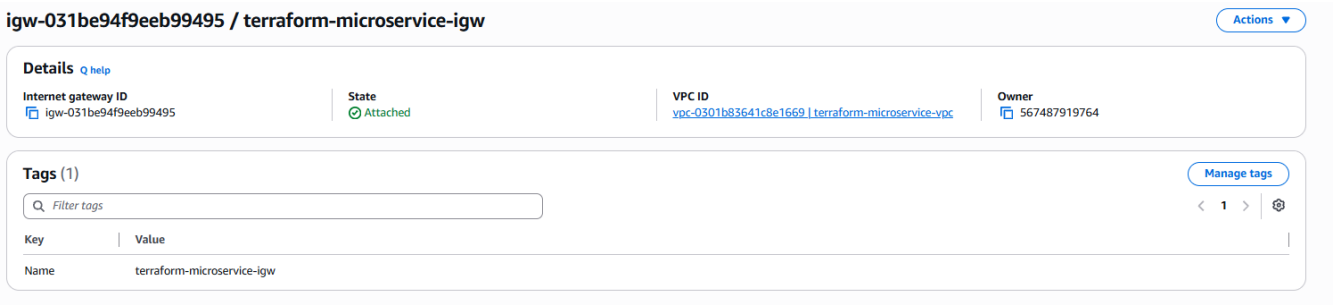
Capture: AWS Console showing the terraform-microservice VPC and 10.0.0.0/16 CIDR.

SCREENSHOT 06 — Public Subnets in Two Availability Zones



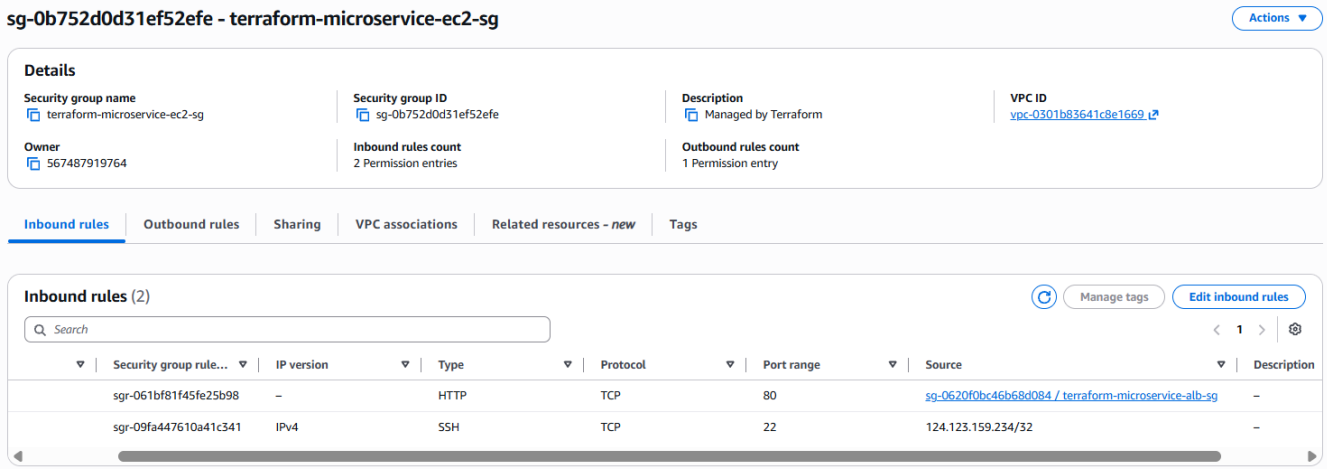
Capture: AWS Console showing both public subnets, their CIDR blocks and Availability Zones.

SCREENSHOT 07 — Internet Gateway and Route Table



Capture: AWS Console showing the Internet Gateway attachment and public route table default route 0.0.0.0/0 to the Internet Gateway.

SCREENSHOT 08 — Security Group Rules



Capture: AWS Console showing ALB HTTP 80 from the internet and EC2 HTTP 80 from ALB SG plus SSH 22 from the administrator /32.

Stage 5 — Generate SSH key and create EC2 infrastructure

6.23 Generate the SSH key locally

```
ssh-keygen -t ed25519 -f terraform-microservice-key -C "terraform-microservice"
```

A passphrase was left empty for this lab. The command created:

```
terraform-microservice-key
terraform-microservice-key.pub
```

The private key is used for SSH authentication and must be kept secure. The public key is imported into AWS.

6.24 Configure .gitignore

File: .gitignore

```
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
*.pem
terraform-microservice-key
terraform-microservice-key.pub
```

This excludes Terraform working files, state/plan files and the local SSH key files from Git.

6.25 Docker installation script

File: scripts/install_docker.sh

```
#!/bin/bash
set -euxo pipefail

dnf install -y docker awscli
systemctl enable --now docker
```

6.26 Container deployment script

File: scripts/deploy_container.sh

```
#!/bin/bash
set -euxo pipefail

aws ecr get-login-password --region ${aws_region} \
  | docker login --username AWS --password-stdin ${registry}

docker pull ${image_uri}

docker run -d \
  --restart unless-stopped \
  --name terraform-microservice \
  -p 80:80 \
  ${image_uri}
```

The deployment script authenticates Docker to ECR, pulls the image and starts the container on port 80. The placeholders are populated by Terraform templatefile().

6.27 Compute module main.tf

File: modules/compute/main.tf

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-kernel-*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_key_pair" "this" {
  key_name   = "${var.project_name}-key"
  public_key = var.public_key
}

resource "aws_iam_role" "ec2" {
  name = "${var.project_name}-ec2-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
```

```
        Service = "ec2.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecr_read" {
  role       = aws_iam_role.ec2.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

resource "aws_iam_instance_profile" "this" {
  name = "${var.project_name}-instance-profile"
  role = aws_iam_role.ec2.name
}

resource "aws_instance" "this" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = var.instance_type
  subnet_id                   = var.subnet_id
  vpc_security_group_ids      = [var.security_group_id]
  key_name                    = aws_key_pair.this.key_name
  iam_instance_profile        = aws_iam_instance_profile.this.name
  associate_public_ip_address = true

  user_data = join("\n", [
    file("${path.module}/../../scripts/install_docker.sh"),
    templatefile("${path.module}/../../scripts/deploy_container.sh", {
      aws_region = var.aws_region
      registry   = split("/", var.ecr_repository_url)[0]
      image_uri  = "${var.ecr_repository_url}:v1"
    })
  ])

  user_data_replace_on_change = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    volume_type = "gp3"
    volume_size = 8
    encrypted   = true
  }

  tags = {
    Name = "${var.project_name}-ec2"
  }

  depends_on = [aws_iam_role_policy_attachment.ecr_read]
}
```

The AMI data source selects the latest matching Amazon Linux 2023 x86_64 image. The key pair imports the local public key. The IAM role is assumed by EC2 and receives the AmazonEC2ContainerRegistryReadOnly managed policy. The instance profile attaches the role to EC2. User data runs the two scripts at first boot. IMDSv2 is required and the 8 GB GP3 root volume is encrypted.

6.28 Compute module variables

File: modules/compute/variables.tf

```
variable "project_name" {
  type = string
}

variable "instance_type" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "security_group_id" {
  type = string
}

variable "public_key" {
  type = string
}

variable "aws_region" {
  type = string
}

variable "ecr_repository_url" {
  type = string
}
```

6.29 Compute module outputs

File: modules/compute/outputs.tf

```
output "instance_id" {
  value = aws_instance.this.id
}

output "public_ip" {
  value = aws_instance.this.public_ip
}

output "public_dns" {
  value = aws_instance.this.public_dns
}
```

6.30 Connect compute module

Add this block to the root main.tf:

```
module "compute" {
  source = "./modules/compute"

  project_name      = local.name_prefix
  instance_type     = var.instance_type
  aws_region        = var.aws_region
  subnet_id         = module.networking.subnet_ids[0]
```

```
  security_group_id = module.networking.ec2_security_group_id
  public_key        = file("${path.module}/terraform-microservice-key.pub")

  ecr_repository_url = module.ecr.repository_url

  depends_on = [terraform_data.push_image]
}
```

The compute module depends on the image-push operation, so the image is pushed before EC2 is created.

6.31 Root networking and compute outputs

Add the following to root outputs.tf, retaining the ECR repository output:

```
output "vpc_id" {
  value = module.networking.vpc_id
}

output "public_subnet_ids" {
  value = module.networking.subnet_ids
}

output "ec2_instance_id" {
  value = module.compute.instance_id
}

output "ec2_public_ip" {
  value = module.compute.public_ip
}

output "ec2_public_dns" {
  value = module.compute.public_dns
}
```

6.32 Validate and review EC2 plan

```
terraform fmt -recursive
terraform validate
terraform plan
```

The reviewed plan showed 5 to add, 0 to change and 0 to destroy. The five resources were the EC2 instance profile, IAM role, IAM role policy attachment, EC2 instance and EC2 key pair. The plan showed the ECR image URI, selected Amazon Linux 2023 AMI, t3.micro instance type, imported key name and user-data commands.

6.33 Create EC2

```
terraform apply
```

After reviewing the plan, type yes. Terraform created the EC2 instance and IAM resources.

6.34 Retrieve EC2 details

```
terraform output
```

The outputs provide the EC2 instance ID, public IP, public DNS, VPC ID and public subnet IDs.

6.35 Connect to EC2 using SSH

```
chmod 400 terraform-microservice-key
ssh -i terraform-microservice-key ec2-user@YOUR_EC2_PUBLIC_IP
```

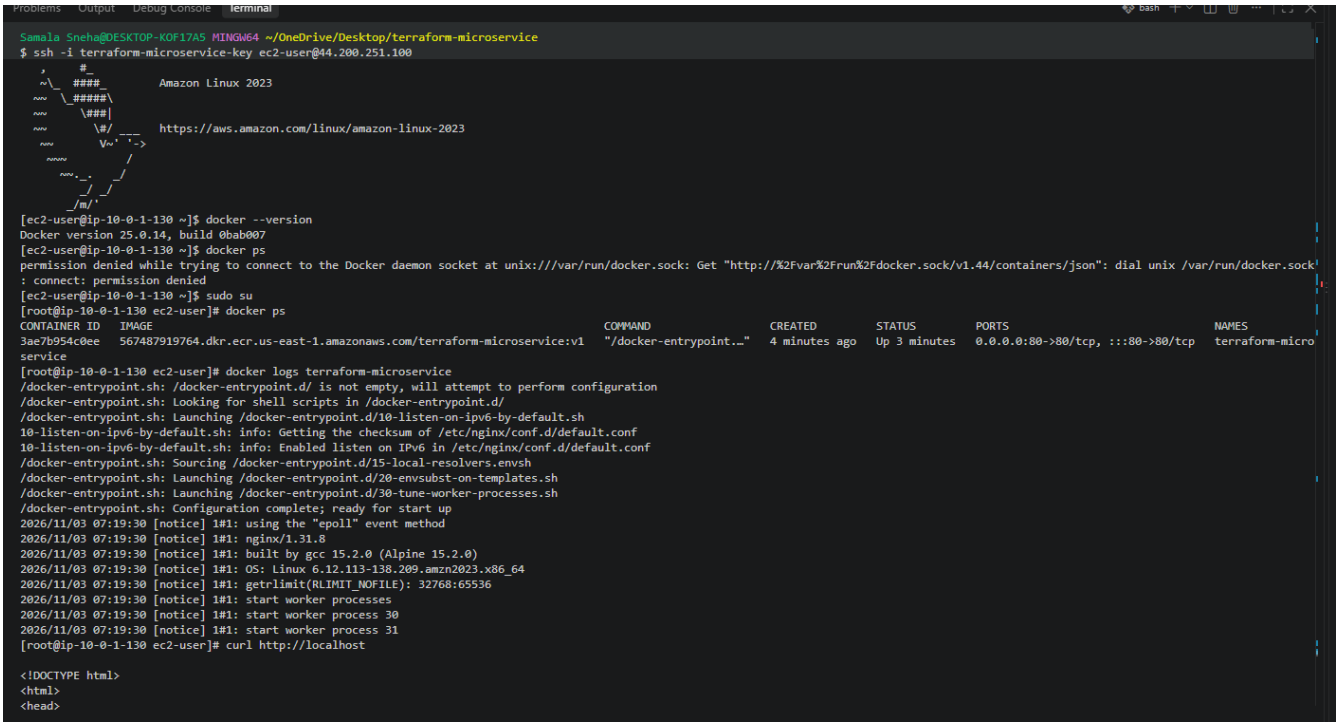
Replace YOUR_EC2_PUBLIC_IP with the value returned by Terraform.

6.36 Verify Docker and application on EC2

```
docker --version
docker ps
docker logs terraform-microservice
curl http://localhost
```

The Docker version was displayed, the application container was running, and curl returned the expected HTML page.

SCREENSHOT 09 — EC2 Instance Running, SSH Connection to EC2, Docker Container Running on EC2, Application Response from EC2



Capture: AWS Console showing the EC2 instance in the running state, Git Bash terminal showing a successful SSH connection as ec2-user, EC2 terminal showing the terraform-microservice container in docker ps.

Stage 6 — Create the Application Load Balancer

6.37 Load balancer module main.tf

File: modules/loadbalancer/main.tf

```
resource "aws_lb" "this" {
  name               = "${var.project_name}-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [var.security_group_id]
  subnets            = var.subnet_ids

  tags = {
```

```
    Name = "${var.project_name}-alb"
  }
}

resource "aws_lb_target_group" "this" {
  name     = "${var.project_name}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = var.vpc_id

  health_check {
    enabled             = true
    path                = "/"
    protocol            = "HTTP"
    port                = "traffic-port"
    matcher             = "200"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 2
    unhealthy_threshold = 2
  }

  tags = {
    Name = "${var.project_name}-tg"
  }
}

resource "aws_lb_target_group_attachment" "this" {
  target_group_arn = aws_lb_target_group.this.arn
  target_id        = var.instance_id
  port             = 80
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.this.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.this.arn
  }
}
```

6.38 Load balancer module variables

File: modules/loadbalancer/variables.tf

```
variable "project_name" {
  type = string
}

variable "security_group_id" {
  type = string
}

variable "subnet_ids" {
  type = list(string)
}
```

```
variable "vpc_id" {
  type = string
}

variable "instance_id" {
  type = string
}
```

6.39 Load balancer module outputs

File: modules/loadbalancer/outputs.tf

```
output "alb_dns_name" {
  value = aws_lb.this.dns_name
}

output "alb_zone_id" {
  value = aws_lb.this.zone_id
}

output "alb_arn" {
  value = aws_lb.this.arn
}

output "target_group_arn" {
  value = aws_lb_target_group.this.arn
}
```

6.40 Connect load balancer module

Add this block to the root main.tf:

```
module "loadbalancer" {
  source = "./modules/loadbalancer"

  project_name      = local.name_prefix
  security_group_id = module.networking.alb_security_group_id
  subnet_ids        = module.networking.subnet_ids
  vpc_id            = module.networking.vpc_id
  instance_id       = module.compute.instance_id
}
```

6.41 Root ALB outputs

Add the following to root outputs.tf:

```
output "alb_dns_name" {
  description = "Public DNS name of the Application Load Balancer"
  value       = module.loadbalancer.alb_dns_name
}

output "application_url" {
  description = "Application URL through the ALB"
  value       = "http://${module.loadbalancer.alb_dns_name}"
}

output "target_group_arn" {
  description = "ARN of the ALB target group"
  value       = module.loadbalancer.target_group_arn
}
```

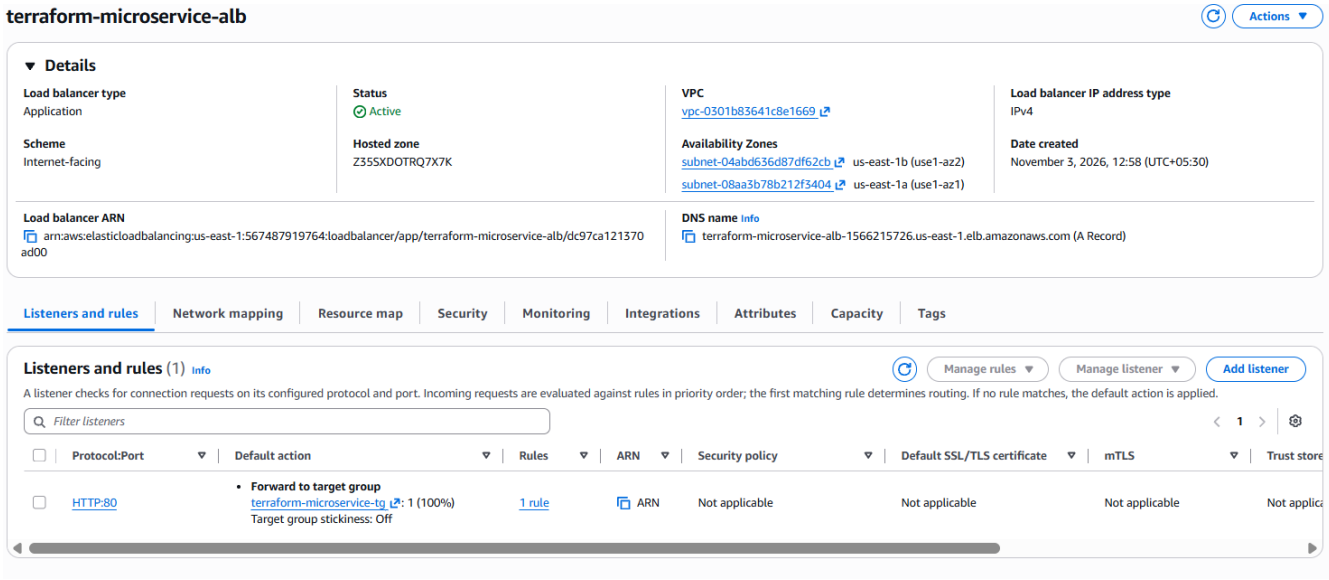
The target_group_arn root output was added after an initial CLI verification attempt reported that the output was not present in Terraform state. Running terraform apply after adding the root output updated the state outputs.

6.42 Validate, plan and apply ALB

```
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
```

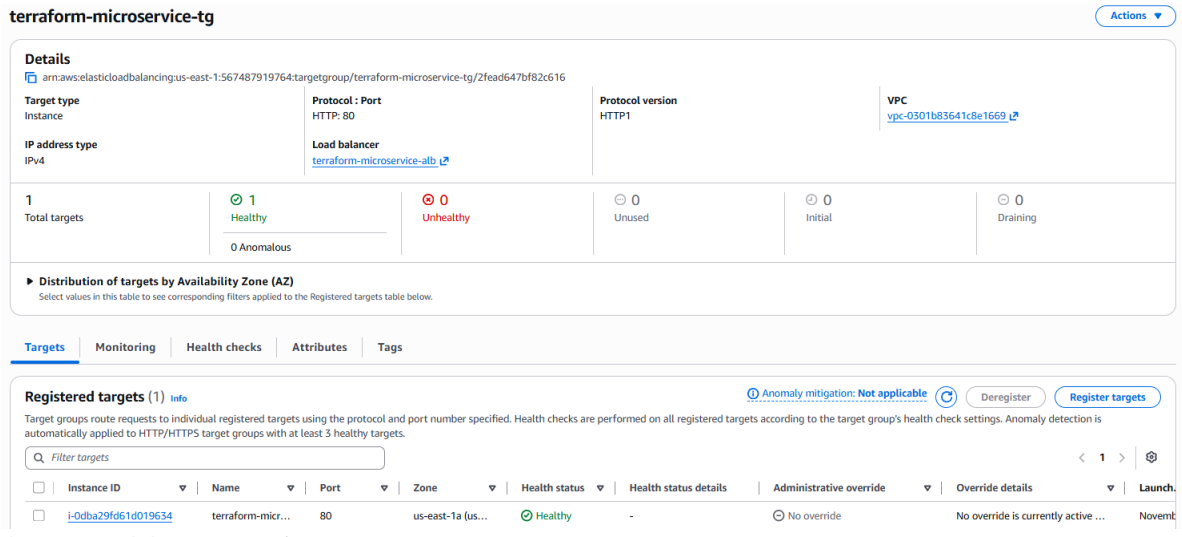
The ALB plan was reviewed before applying. It created the Application Load Balancer, target group, target group attachment and HTTP listener.

SCREENSHOT 10 — Application Load Balancer Created



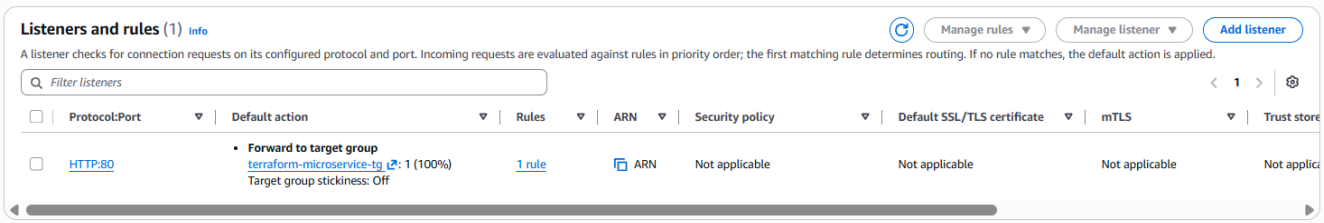
Capture: AWS Console showing the internet-facing ALB in the active state.

SCREENSHOT 11 — Target Group Configuration



Capture: AWS Console showing target group HTTP port 80 and health check path /.

SCREENSHOT 12 — ALB Listener Configuration



Capture: AWS Console showing the HTTP port 80 listener forwarding to the target group.

Stage 7 — Verify ALB health and application access

6.43 Retrieve Terraform outputs

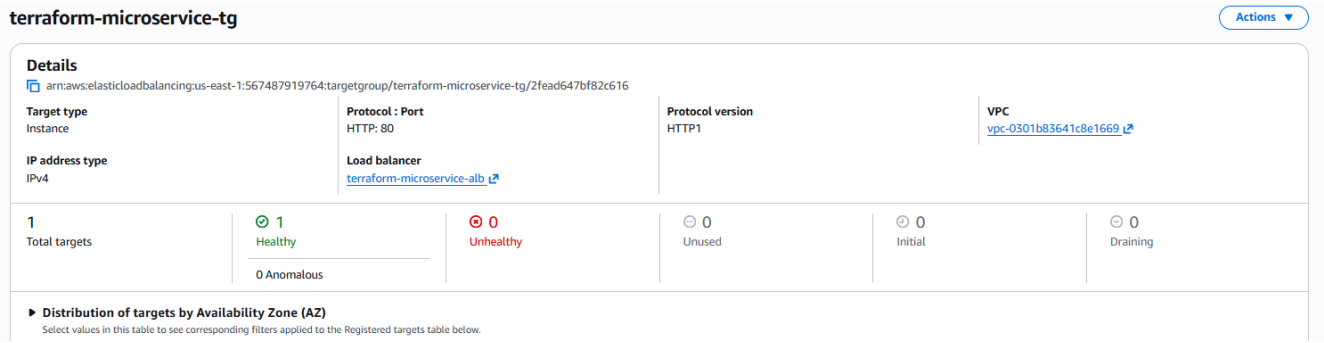
```
terraform output
terraform output -raw application_url
terraform output -raw target_group_arn
```

6.44 Verify target health

```
aws elbv2 describe-target-health \
  --target-group-arn "$(terraform output -raw target_group_arn)" \
  --region us-east-1
```

The target was checked in the AWS Console and displayed Healthy. The target group health check uses HTTP port 80, path /, and expects HTTP 200.

SCREENSHOT 13 — Target Group Healthy Status



Capture: AWS Console showing the EC2 target registered and Healthy.

6.45 Verify application through ALB

Retrieve the application URL:

```
terraform output -raw application_url
```

Open the returned URL in a browser. The application page was displayed successfully through the ALB DNS name.

SCREENSHOT 14 — Application Accessible Through ALB DNS



Capture: Browser showing the application page using the ALB DNS name.

6.46 Verify HTTP response status

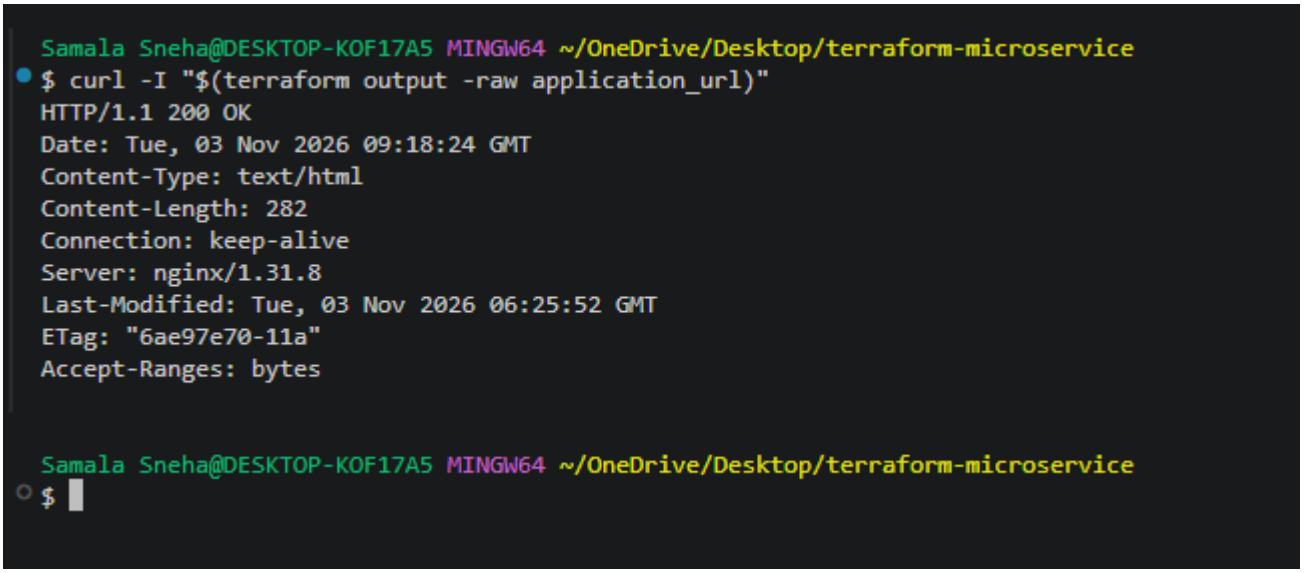
```
curl -I "$(terraform output -raw application_url)"
```

Actual terminal response:

```
HTTP/1.1 200 OK
Date: Tue, 03 Nov 2026 07:38:02 GMT
Content-Type: text/html
Content-Length: 282
Connection: keep-alive
Server: nginx/1.31.8
Last-Modified: Tue, 03 Nov 2026 06:25:52 GMT
ETag: "6ae97e70-11a"
Accept-Ranges: bytes
```

This confirms that the ALB successfully forwarded the request to Nginx.

SCREENSHOT 15 — ALB HTTP 200 Verification



Capture: Git Bash showing the curl -I response with HTTP/1.1 200 OK.

6.47 Verify direct EC2 application access is blocked

```
curl --connect-timeout 5 "http://$(terraform output -raw ec2_public_ip)"
```

Actual result:

```
curl: (28) Connection timeout after 5012 ms
```

This is expected. The EC2 security group does not permit direct internet access to application port 80; it accepts port 80 traffic only from the ALB security group.

SCREENSHOT 19 — Direct EC2 Access Blocked

[INSERT YOUR SCREENSHOT HERE]
Capture: Git Bash showing the connection timeout when accessing the EC2 public IP on port 80.

7. Final Security Verification

Requirement	Implementation	Result
Application not directly internet-accessible	EC2 port 80 accepts traffic only from ALB security group.	Verified by direct-access timeout.
Application accessible through ALB	Internet-facing ALB on HTTP port 80.	Verified; HTTP 200.
SSH access restricted	Port 22 limited to administrator /32 CIDR.	Configured; confirm current IP remains correct.
ECR access from EC2	IAM role with AmazonEC2ContainerRegistryReadOnly.	Configured and image pull succeeded.
No static AWS keys on EC2	EC2 instance profile provides role credentials.	Implemented.
Encrypted EC2 root volume	Encrypted 8 GB GP3 root volume.	Configured.
IMDSv2 required	http_tokens = required.	Configured.
ALB health check	HTTP / expecting 200.	Healthy in AWS Console.

8. Final Results

8.1 Deployment summary

Component	Final result
Local Docker image	Built successfully.
Local Docker test	Successful.
ECR repository	Created.
Image push to ECR	Successful.
VPC	Created.
Public subnets	Two created in separate Availability Zones.
Internet Gateway	Created and attached.
Public route table	Created and associated with both subnets.
Security groups	Configured for ALB, application and SSH traffic.
SSH key pair	Generated locally; public key imported into AWS.
EC2	Running Amazon Linux 2023.
IAM role	Attached to EC2.
Docker on EC2	Installed.
ECR image pull	Successful.
Application container	Running.
Application Load Balancer	Created.
Target group	Healthy.
ALB browser access	Successful.
Direct EC2 application access	Blocked as intended.

8.2 Actual verification evidence

Test	Observed result	Outcome
ALB URL	HTTP/1.1 200 OK	Passed
Direct EC2 public IP	Connection timeout after 5012 ms	Passed — direct app access blocked
AWS Console target health	Healthy	Passed
Browser test	Application displayed through ALB DNS	Passed
EC2 local curl	Expected application HTML returned	Passed

9. Important Implementation Concepts Learned

9.1 Terraform modules

Separate modules were used for ECR, networking, compute and load balancing. This separates responsibilities and makes the project easier to maintain and reuse.

9.2 terraform_data and local-exec

The ECR image push uses terraform_data with local-exec. This lets Terraform run local Docker and AWS CLI commands as part of the workflow. triggers_replace controls when the operation is recreated and run again.

9.3 Terraform dependencies

```
depends_on = [terraform_data.push_image]
```

The compute module depends on the image-push operation so EC2 is not created before the ECR image push has completed.

9.4 EC2 user data

EC2 user data installs Docker, authenticates to ECR, pulls the image and starts the application container during instance initialization. This avoids manually installing and deploying the container after launch.

9.5 IAM instance profile

The EC2 instance uses an IAM role through an instance profile instead of storing AWS access keys on the server. The role is attached to the AmazonEC2ContainerRegistryReadOnly managed policy.

9.6 Security groups

The ALB security group allows public HTTP access. The EC2 security group accepts application traffic only from the ALB security group and SSH only from the configured administrator /32 address.

9.7 Application Load Balancer

The ALB accepts incoming HTTP requests and forwards them to registered targets. Health checks determine whether a target is healthy and able to receive traffic.

10. Conclusion

The Terraform-based containerized microservice deployment was successfully completed on AWS. A static web application was created, packaged into a Docker image and tested locally. Terraform created an Amazon ECR repository and pushed the local image into it.

The AWS infrastructure was provisioned using reusable Terraform modules, including a custom VPC, public subnets, Internet Gateway, route table, security groups, EC2 instance, IAM role and Application Load Balancer.

The EC2 instance automatically installed Docker, authenticated to ECR using its IAM role, pulled the application image and started the Nginx container using user data.

The Application Load Balancer was configured with a target group and HTTP listener. The application was verified through the ALB DNS name and returned HTTP 200. Direct access through the EC2 public IP was blocked, confirming the intended network security design.

The assignment's main deployment and verification requirements were achieved successfully using Terraform.

10A. AWS Cost Considerations

The lab was designed to avoid a NAT Gateway, but AWS resources can still incur charges while they exist. In particular:

- An Application Load Balancer is billed while provisioned, even when it receives no traffic.
- The EC2 instance, EBS root volume and any public IPv4 address may incur charges depending on account eligibility and current AWS pricing.
- Amazon ECR can incur image storage and data-transfer charges.
- The t3.micro instance type and any free-tier or promotional eligibility depend on the AWS account and current AWS offer terms; verify the Billing console rather than assuming all usage is free.

Destroy the infrastructure when the assignment is finished and verify that no billable resources remain. The cleanup procedure is provided in the next section.

11. Consolidated Cleanup Procedure

IMPORTANT — Run cleanup only after the assignment is complete and the deployed infrastructure is no longer required. An active ALB and other AWS resources can continue to incur charges.

11.1 Review Terraform-managed resources

```
terraform state list
```

This lists the resources currently tracked by Terraform state.

11.2 Destroy AWS infrastructure

```
terraform destroy
```

Review the destruction plan carefully and type yes to approve. Terraform will remove the managed resources, including:

- Application Load Balancer, listener, target group and target attachment.
- EC2 instance.
- IAM instance profile, IAM role and policy attachment.
- Imported EC2 key pair.
- Security groups.
- Route table and associations.
- Internet Gateway.
- Public subnets and VPC.
- ECR repository and its images (the repository has force_delete = true).

11.3 Verify AWS cleanup

Check the AWS Console and confirm that the EC2 instance is terminated, ALB is deleted, ECR repository is deleted, VPC networking resources are removed, and the IAM role and instance profile are deleted.

11.4 Remove the local test container

```
docker stop terraform-microservice-test
docker rm terraform-microservice-test
```

11.5 Remove the local test image if no longer needed

```
docker rmi terraform-microservice:v1
```

11.6 Remove local SSH keys if no longer needed

```
terraform-microservice-key
terraform-microservice-key.pub
```

Delete these local files only after confirming they are no longer needed. Never delete Terraform state before running terraform destroy, because Terraform needs the state to identify and remove managed resources.

After destruction, review AWS resource inventories and billing. Any resources not tracked in this Terraform state must be checked separately.

Appendix A — Consolidated Root Terraform Files

The following shows how the root main.tf is assembled from the module and image-push blocks documented above. It is included as a consolidated reference; retain the same blocks in your project.

A.1 Root main.tf

```
module "ecr" {
  source = "./modules/ecr"

  repository_name = local.name_prefix
}

resource "terraform_data" "push_image" {
  triggers_replace = [
    filesha256("${path.module}/application/Dockerfile"),
    filesha256("${path.module}/application/index.html"),
    module.ecr.repository_url
  ]

  provisioner "local-exec" {
    interpreter = ["bash", "-c"]

    command = <<-EOT
      set -e

      aws ecr get-login-password --region ${var.aws_region} | docker login --username AWS --password-stdin
${split("/", module.ecr.repository_url)[0]}

      docker tag terraform-microservice:v1 ${module.ecr.repository_url}:v1

      docker push ${module.ecr.repository_url}:v1
    EOT
  }

  depends_on = [module.ecr]
}
```

```
module "networking" {
  source = "./modules/networking"

  project_name = local.name_prefix
  ssh_cidr     = var.ssh_cidr
}

module "compute" {
  source = "./modules/compute"

  project_name      = local.name_prefix
  instance_type     = var.instance_type
  aws_region        = var.aws_region
  subnet_id         = module.networking.subnet_ids[0]
  security_group_id = module.networking.ec2_security_group_id
  public_key        = file("${path.module}/terraform-microservice-key.pub")

  ecr_repository_url = module.ecr.repository_url

  depends_on = [terraform_data.push_image]
}

module "loadbalancer" {
  source = "./modules/loadbalancer"

  project_name      = local.name_prefix
  security_group_id = module.networking.alb_security_group_id
  subnet_ids        = module.networking.subnet_ids
  vpc_id            = module.networking.vpc_id
  instance_id       = module.compute.instance_id
}
```

A.2 Consolidated root outputs.tf

```
output "ecr_repository_url" {
  description = "Amazon ECR repository URL"
  value       = module.ecr.repository_url
}

output "vpc_id" {
  value = module.networking.vpc_id
}

output "public_subnet_ids" {
  value = module.networking.subnet_ids
}

output "ec2_instance_id" {
  value = module.compute.instance_id
}

output "ec2_public_ip" {
  value = module.compute.public_ip
}

output "ec2_public_dns" {
  value = module.compute.public_dns
}
```

```
output "alb_dns_name" {
  description = "Public DNS name of the Application Load Balancer"
  value       = module.loadbalancer.alb_dns_name
}

output "application_url" {
  description = "Application URL through the ALB"
  value       = "http://${module.loadbalancer.alb_dns_name}"
}

output "target_group_arn" {
  description = "ARN of the ALB target group"
  value       = module.loadbalancer.target_group_arn
}
```